3S – Secure Streaming Solution (Part I): Secure Channel, Handshake and Flexible Cryptographic Configurations for Real-Time MPEG-4 Streaming

Student 79449 and Student 72966

Computer Systems Security, 2026/2027, 1st Semester – Work Assignment #1 – Pair Programming

1 Introduction

The provided baseline consists of an OriginSocketServer (Stream Server) that serves MP4 files in 1 KiB blocks over raw TCP sockets and a ProxySocketServer (Proxy) that relays them to a browser, honoring HTTP Range requests. The baseline offers no secrecy, integrity, peer authentication or replay protection.

The objective of Work Assignment 1 is to build a secure channel between Proxy and Stream Server with confidentiality, message integrity and authenticity, replay protection and forgery protection, negotiated by a handshake and fully configurable without recompilation. Our work now covers the whole chain: the cryptographic configuration layer, the handshake, the protected message channel, the integration of all of them into the Secure Proxy and the Secure Stream Server (with the metrics required by the assignment), and an adversarial test bench that attacks both the isolated channel and the running system. The authenticated Diffie-Hellman key agreement and the optional extensions are still pending.

2 System Model and Architectural Design

The browser speaks plain HTTP to the local Secure Proxy (which runs on the viewer's trusted host). The Proxy opens a TCP connection to the Secure Stream Server and runs the SSP over it. The Authentication Server belongs to Work Assignment 2 and is not part of this deliverable.

The shared library contains CryptoConfig (parsing and validation), CryptoSuite (encryption/decryption of one message), Handshake (types 1–3), SecureChannel (framing, types 4/5/255, sequence numbers, AAD) and typed exceptions mapped to alert codes. Two thin adapters connect the channel to the baseline code: SecureOutputStream (used by the Server; every write becomes a type 4 message and close() becomes the type 5 end block) and SecureInputStream (used by the Proxy; every read returns decrypted, verified data and returns -1 only after a valid type 5). Because of these adapters the baseline HTTP parsing, Range handling and block loops remain almost unchanged.

Table 1: Implementation status at the time of writing.

Component	Content	Status
CryptoConfig	Parsing (hardened against XXE), hex/Base64, validation, including the 256-bit key rule for ChaCha20	Done, tested
CryptoSuite	AEAD and Encrypt-then-MAC, random nonce per message, AAD	Done, tested

Component	Content	Status
Handshake	Types 1–3, SRV challenge, config confirmation	Done, tested
SecureChannel	Header framing, types 4/5/255, SEQ checks, byte counters	Done, tested
Secure Proxy / Server	HTTP↔SSP integration, Range handling, 404 handling, alerts	Done, tested with browser, curl and a live attacker
Metrics	Blocks received/forwarded, media bytes, network bytes, throughput	Done, measured for all five configurations
Attack tools	StreamMallory (live man-in-the-middle), ChannelTest, HandshakeTest	Done
DH key agreement	Authenticated ephemeral key exchange, forward secrecy	Pending (preshared keys)
Extensions	Concurrency stress tests, Docker	Pending

3 Threat Model and Security Countermeasures

A network attacker ("Mallory") controls the path between Proxy and Server: it can eavesdrop, modify, inject, replay, reorder, duplicate and drop messages, and can open it own connections to the Server. Both endpoints are uncompromised; the primitives (AES, ChaCha20, Poly1305, GCM, HMAC-SHA2, SHA-256) are secure when correctly used; the configured keys are secret and random. Endpoint compromise, breaking primitives and traffic analysis (message sizes and timing) are out of scope; implementation errors, bad configurations and protocol misuse are in scope.

Table 2 maps threaten mechanisms. We report honestly the requirements that the present implementation does not meet.

Table 2: Threat-to-mechanism mapping.

Threat	Mechanism	Status
Eavesdropping on media	AEAD or cipher+HMAC; fresh random nonce per message. The HTTP request, response headers, Range and movie name also travel inside the encrypted channel	Mitigated (confirmed on a live stream, Sect. 7.4)
Tampering with a block	GCM/Poly1305 tag or HMAC over aad‖nonce‖ct; any bit flip rejected (INTEGRITY_FAILURE)	Mitigated
Forgery	Requires the secret key(s); verification before use (HMAC checked before decryption; constant-time	Mitigated (given key

Threat	Mechanism	Status
	compare)	secrecy)
Replay / duplication	Receiver accepts only the next expected SEQ; SEQX random (62 bit); fresh 256-bit SRV per handshake; R = SHA-256(SRV)	Mitigated
Reordering, dropping	Strict SEQ+1 rule; SEQ also in AAD, so rewriting the clear SEQ fails the tag	Mitigated
Truncation / block-END confusion	Authenticated end block (type 5); EOF before it is an error; version, release, type and SEQ in AAD	Mitigated (see note below the table)
Altered / downgraded proposal	Server accepts only a configuration identical to its own; confirmation CCC returned inside encrypted type 2 and compared by the Proxy	Mitigated
Tampering with HTTP metadata (status, Content-Length, Content-Range, Range)	Carried inside type 3 (Range) and inside the encrypted type 4 stream (response headers)	Mitigated
Weak configuration (ECB, no MAC, same key for cipher and MAC, wrong key size)	Rejected at load time by validate()	Mitigated
Resource exhaustion by forged size	Payload size capped at 1 MiB before allocation	Mitigated
Path traversal through the movie name	Server resolves the name against the media directory and answers 404 for anything outside it or not a regular file	Implemented; no dedicated test yet
Man-in-the-middle on key agreement	Peers authenticated only implicitly, by possession of the pre-shared key. No DH exchange yet	Partial
Session-key compromise / forward secrecy	Keys are static: a leaked key exposes all recorded and future sessions	Not met
Forged alerts (type 255) / connection reset	Alerts are unauthenticated; an on-path attacker can already cut the connection	Out of scope
User authentication	Authentication Server	WA2

4 Specification of the Secure Streaming Protocol

Every message is Header‖Payload. The header is clear: version (2 B), release (2 B, currently 1:1), type (1 B) and payload size (4 B), i.e. 9 bytes. Sequence numbers are 8 bytes. E_K(·) denotes the configured protection of one message (Sect. 5); aad_t = ver‖rel‖t for type t.

Handshake: The Proxy sends SEQX and the proposal PCC taken from proxycrypto.config. The Server compares it with sservercrypto.config; if incompatible, it answers alert CONFIG_REJECTED. Otherwise, it returns, encrypted, SEQY=SEQX+1, a 256-bit nonce SRV from SecureRandom and the confirmation of CCC. The Proxy checks the sequence number and that CCC equals what it proposed (type 1 travels in clear, so this encrypted echo is what exposes a modified proposal). Message 3 carries SEQZ=SEQY+1, R=SHA-256(SRV) and the MEDIA field; the Server verifies R in constant time and only then streams.

MEDIA field: Since the browser issues Range requests, MEDIA is the movie name followed by CR LF and the browser Range header (empty if none). It is encrypted and authenticated as part of type 3, so Mallory can neither learn which movie is watched nor alter the requested range.

Streaming: Blocks are sent as SEQB‖E_K(block) with SEQB starting at SEQZ+1, with the clear SEQB also bound in the AAD (aad_t‖SEQB). The receiver checks the SEQ first and then authenticates and decrypts. The first type 4 message carries the HTTP status line and headers produced by the Server (status, Content-Length, Content-Range, Content-Type), so they are protected like the media; the following type 4 messages carry the file in 1 KiB blocks. The last message is type 5 with SEQF and an encrypted body SEQF‖“END”‖random. SEQF is therefore both bound in the AAD and repeated in the plaintext, and the receiver checks both. A connection that ends before a valid type 5 is an error. The Server also sends the type 5 after a 404 answer, so even an error response is a complete, authenticated stream.

Alerts: Type 255 carries a code (1 byte) and a reason: 1 unexpected message, 2 bad version, 3 malformed, 4 integrity failure, 5 bad sequence, 6 configuration rejected, 7 bad challenge, 8 media not found. Alerts 1–7 are used as described. In the implementation, a missing movie is reported as an HTTP 404 inside the authenticated stream (Sect. 6), so alert 8 is defined but not raised. Decryption failures always return the same message, to avoid giving an oracle to the attacker.

5 Cryptographic Configuration Files

Both proxycrypto.config and sservercrypto.config follow the XML layout of the statement (SymCipherSuite, MACsuite, ConfiguredKeys); NULL marks an unused field. Nothing cryptographic is hard-coded: the JCE transformation, IV size, MAC name and keys are all derived from the file. Tag names are matched case-insensitively, and keys are accepted in hex or Base64 with their declared size checked. A small tool (GenerateKey) produces keys of the right size for each algorithm, and ConfigTest prints how each file is interpreted (transformation, proposal string, provider, MAC length) or why it is refused.

Message protection: AEAD modes (GCM, Poly1305, CCM) [1, 2, 3] output nonce‖ct‖tag with the AAD passed to the cipher. Other modes use Encrypt-then-MAC with an independent key: nonce‖ct‖HMAC_K2(aad‖nonce‖ct) [4]. A fresh random nonce is drawn for every message. Five configurations are provided and tested (Table 3); the IV size is taken from the file.

Validation at load time: The loader refuses: ECB; any non-AEAD mode without a MAC; a MAC without a key; a MAC key equal to the encryption key; key sizes other than 128/192/256 bits; a key whose length

disagrees with its declared size; a mode with no IV size; and ChaCha20 with a key that is not 256 bits. Five invalid example files (examples/INVALIDO-*.config) exercise these rules, and all five are now refused at load time with a specific message. The ChaCha20 case with a 128-bit key, which an earlier version only detected at first use, is now detected when the file is loaded.

6 Secure Proxy and Stream Server: Design and Implementation

Proxy: The proxy loads and validates proxycrypto.config at start-up and prints the active proposal. For each browser request it parses the HTTP request line and the Range header, opens a TCP connection to the Server and runs Handshake.proxy() with the movie name and Range as MEDIA. It then wraps the channel in a SecureInputStream and reuses the baseline code to read the status line and headers (now decrypted) and to forward the body block by block to the browser. After the last byte of Content-Length, it requires a valid type 5; any extra data is a protocol error. A protocol violation sends the corresponding alert to the Server and aborts the stream; if no HTTP header has been sent yet, the browser receives 502 Bad Gateway. The browser therefore never receives an unauthenticated byte: each block is written to it only after it has been verified and decrypted. Browsers issue several Range requests (seeking, buffering), so each upstream request is a separate SSP session with its own handshake, SRV and SEQX, which keeps replay protection per session.

Server: It accepts a connection, runs Handshake.server() against sservercrypto.config, splits MEDIA into movie name and Range, and replaces its output stream with a SecureOutputStream starting at the first block SEQ returned by the handshake. The baseline serveFile and header code then run unchanged, so Range handling (200, 206, 416) continues to work. The name is resolved against the media directory, and anything outside it, or not a regular file, gets a 404 sent through the secure stream. The stream is closed in a final block, which emits the authenticated type 5 in every case. Protocol errors are logged and answered with an alert. The baseline already used a cached thread pool, and the CryptoSuite and counters are created per connection, so concurrent sessions are independent.

SecureChannel counts the bytes read from the wire, and ProxyMetrics prints a report after every stream (every GET /video/… is one stream). It reports the HTTP status, whether the stream ended with a verified END, the number of encrypted blocks received, the number of decrypted blocks forwarded, the plaintext media bytes forwarded compared with the size asked and with the original file, the bytes received from the network, and the throughput. Throughput is the plaintext media bytes divided by the time from the end of the handshake (System.nanoTime) until the END is verified, or the stream stops, with 1 kB = 1000 bytes. The number of encrypted blocks is one higher than the number of forwarded blocks because the first type 4 message carries the HTTP headers: for the 25,941,346-byte test file there are 25,335 encrypted blocks, of which 25,334 carry media.

7 Validation

7.1 Testbench

The tests are Java programs in common/ and tools/, with raw outputs saved in results/.

CryptoSuiteTest: round-trip, nonce uniqueness, exhaustive single-bit/AAD/truncation tampering and per-1 KiB overhead. **ConfigTest:** loads every example file and prints the interpretation or the refusal. **ChannelTest (per configuration):** a recorded session (type 2, three type-4 blocks, type 5) is manipulated

by Mallory and fed to a receiver. **HandshakeTest:** real TCP sockets with a relay in the middle that applies to the attack. **StreamMallory:** a live man-in-the-middle placed between the real Proxy and the real Server (the Proxy's origin port points to Mallory) that applies one attack to the N-th type 4 message (default 50). End-to-end runs use a real browser and curl against the Proxy, with a 25.9 MB MP4 (dante.mp4) on the Server. All tests use the five example configurations, or the default AES-GCM-256.

7.2 Results: cryptographic suite and channel

Cryptographic suite: For all five configurations the round trip succeeds, each message gets a new nonce and every modification is rejected. Table 3 lists the measured expansion per 1 KiB message and the per-block wire cost (adding the 9 B header and 8 B SEQ).

Table 3: Tested configurations: measured expansion of a 1 KiB message and derived per-block wire overhead.

Configuration	Nonce/IV	Suite expansion	Wire overhead per block
AES/GCM/NoPadding (96-bit IV)	12 B	+28 B (2.7%)	45 B (4.4%)
ChaCha20-Poly1305	12 B	+28 B (2.7%)	45 B (4.4%)
ChaCha20 + HMAC-SHA256	12 B	+44 B (4.3%)	61 B (6.0%)
AES/CTR + HMAC-SHA256	16 B	+48 B (4.7%)	65 B (6.3%)
AES/CBC/PKCS5 + HMAC-SHA-512	16 B	+96 B (9.4%)	113 B (11.0%)

Channel attacks: In each of the five configurations, all 12 scenarios behave as required (60/60). The normal session is accepted; a replayed, swapped, dropped or injected-from-another-session block is rejected with BAD_SEQUENCE; a dropped block whose clear SEQs were "repaired", a block turned into an end-of-stream (type 4→5) and a single flipped ciphertext bit are rejected with INTEGRITY_FAILURE; a changed version gives BAD_VERSION; a forged 2 GB size gives MALFORMED (no allocation); and a missing end block or cut connection is detected as premature end of stream. The "repaired SEQ" case shows the value of binding SEQ in the AAD.

7.3 Results: handshake

Handshake (AES-GCM, real sockets): All 7 scenarios pass: normal handshake (both sides obtain the same first-block SEQ and movie name); a Proxy proposing ChaCha20/Poly1305 against an AES-GCM Server (CONFIG_REJECTED); same configuration but different key (INTEGRITY_FAILURE); Mallory altering the PCC (CONFIG_REJECTED) or SEQX (BAD_SEQUENCE); replay of old messages 1 and 3 (BAD_CHALLENGE, since SRV is fresh); and message 3 sent first (UNEXPECTED_MESSAGE).

The same behavior was confirmed end to end with the real executables. A Server configured with AES-GCM was contacted by Proxies configured with AES-CTR, AES-CBC, ChaCha20-Poly1305 and ChaCha20+HMAC. Both sides logged CONFIG_REJECTED, the browser received no media (0 bytes), and the Proxy reported HTTP error status. A Proxy with the right algorithm but a different key (aes-gcm-

outra-chave.config) produced INTEGRITY_FAILURE on both sides in each of three attempts, again with no media forwarded.

7.4 Results: end-to-end streaming and live attacks

Full stream in all five configurations: Table 4 shows one complete download of the 25,941,346-byte file through the browser-facing Proxy in each configuration. All five ended with "COMPLETE (END verified)", and the media bytes forwarded equal the original file size. The measured network overhead matches the per-block cost derived in Table 3 (4.4%, 4.4%, 6.0%, 6.3%, and 11.0%).

Table 4: End-to-end full-file stream, one run per configuration (localhost).

Configuration	Network bytes received	Overhead vs. file	Time	Throughput
AES/GCM/NoPadding	27,081,717	4.40%	1.573 s	16.5 MB/s
ChaCha20-Poly1305	27,081,722	4.40%	1.256 s	20.7 MB/s
ChaCha20 + HMAC-SHA256	27,487,110	5.96%	1.362 s	19.0 MB/s
AES/CTR + HMAC-SHA256	27,588,458	6.35%	1.843 s	14.1 MB/s
AES/CBC/PKCS5 + HMAC-SHA-512	28,804,609	11.04%	2.336 s	11.1 MB/s

These throughput figures should be read as indicative only. Each is a single cold run on one machine over the loopback interface, and the consumer on the other side is the browser or curl, so JVM warm-up and the client matter. The AES-GCM stream relayed through Mallory in later runs reached 33 and 63 MB/s on the same file, so the differences among the cheaper configurations are within run-to-run noise. The only clear trend is that AES-CBC with HMAC-SHA-512 is the slowest, consistent with having both the largest overhead and two passes over the data. All configurations are fast enough for real-time playback of the test video.

Browser playback and Range requests: With AES-GCM and with AES-CTR+HMAC, the browser played the video and seeked in it, generating 59 and 60 independent sessions respectively, each with its own handshake. Each was a Range request answered with 206. One of them, the final range of the file, ended with a verified END and exactly the bytes asked (152,930). The others show "INCOMPLETE" with a connection-reset exception, which is the browser abandoning a request when it seeks or has buffered enough; the Proxy stops cleanly, and no integrity error is raised. Requesting a file that does not exist (sample.mp4, the default name in the test page) returned HTTP 404 inside a complete, authenticated stream of a single block. No cross-session errors appeared in any of these runs; a dedicated concurrency stress test remains to be done.

Live attacks with StreamMallory: Mallory sits between the real Proxy and the real Server and attacks the 50th type 4 message, which is the 49th block of media since the first message holds the HTTP headers. Table 5 gives the result of each attack as seen by the Proxy and by curl.

Table 5: Live attacks (AES-GCM, attack on message 50).

Attack	Proxy outcome	Media bytes delivered to the client
none (relay only)	COMPLETE, END verified	25,941,346 (all)
eavesdrop	COMPLETE, END verified	25,941,346 (all)
flip one byte of the ciphertext	INTEGRITY_FAILURE	49,152
drop the message	BAD_SEQUENCE (reordered or missing)	49,152
send the message twice	BAD_SEQUENCE (replayed or duplicated)	50,176
swap messages 50 and 51	BAD_SEQUENCE (reordered or missing)	49,152
change type 4 into type 5 in the header	INTEGRITY_FAILURE	49,152
inject a forged message with the next SEQ	INTEGRITY_FAILURE	49,152
remove the type 5 end message	INCOMPLETE (premature end of stream)	25,941,346 (all, no end marker)

In every active attack that touches a data block, the Proxy delivers only the blocks that arrived intact before the attack (48 media blocks = 49,152 bytes, or 49 = 50,176 bytes in the duplicate case, where the first copy is genuine), raises the matching alert and closes the connection. The client still receives HTTP status 200 because the headers had already been forwarded, but the body is far shorter than the announced Content-Length, so the truncation is visible to the browser. The result is the same as the offline ChannelTest, now on real traffic. In the eavesdrop run Mallory captured the first megabyte in each direction (125 bytes Proxy→Server and about 1.05 MB Server→Proxy) and searched for the movie name, the request line, the HTTP response line, the Content-Length header and the MP4 “ftyp” signature. None was found, so the secure channel hides the file content and the request metadata that the baseline sent in clear. The “cut” case is the one described in the note of Sect. 3: all media was delivered, and the missing end block is detected and logged by the Proxy.

8 Conclusions and Future Work

We specified and implemented 3S Part I end to end: a configurable cipher-suite layer, an SSP handshake that detects modified proposals, wrong keys and replays, a block channel that detects tampering, forgery, replay, reordering, dropping and truncation, and a Secure Proxy and Stream Server that stream a real MP4 file to a browser through this channel, including seeking with Range requests. All 60 + 7 + 10 recorded offline tests pass, all five configurations stream the complete test file with a verified end block, and nine live runs with a man-in-the-middle confirm that eavesdropping reveals nothing recognisable and that every active attack is detected before unauthenticated data reaches the browser. The measured overhead (4.4% to 11.0%) follows the cost predicted from the message format.

Remaining work:

- The security gap that matters most is the key agreement. Keys are pre-shared and static, so peers are authenticated only implicitly, and a leaked key exposes every recorded session. An authenticated ephemeral Diffie-Hellman exchange in the handshake would add forward secrecy and per-session keys.
- A dedicated test for concurrent sessions, and for path traversal in the movie name.
- Repeating the throughput measurements with warm-up and several runs per configuration, ideally with a non-browser client.
- Optional extensions (Docker packaging).